Retry Pattern

Category: Resilience **One-liner:** Automatically re-attempt operations that failed for temporary reasons, spacing attempts out with backoff and randomness, so brief glitches don't become user-visible errors.

> This chapter is the book's home for **idempotency keys**, **jitter / the thundering herd**, and the **"retry at one layer only"** rule; other chapters link here.

Part 0 — Concepts you need first

What is a "transient" failure?

Transient = temporary; it clears without anyone fixing anything. **Permanent** = fails identically however often you try. The single most important idea here: **retrying a transient failure helps; retrying a permanent one is pure waste.**

Transient (retry helps)	Permanent (retry is useless)
Network dropped a packet	The password is wrong
Server briefly out of connections	The request body is malformed
Momentary CPU spike	You lack permission
Load balancer restarted an instance	The record doesn't exist

(A **packet** is one chunk of network data; the **request body** is the data sent with a request, e.g. a JSON order.)

What is "backoff"?

Waiting between attempts instead of retrying instantly. **Exponential backoff** makes each wait longer, typically doubling: 1s, 2s, 4s, 8s. Why wait? Retrying instantly adds **load** to a struggling service at the worst moment and gives it no time to recover.

What is "jitter"?

Deliberate randomness in the wait: instead of every client waiting exactly 4 s, each waits a random time around 4 s. It solves the thundering herd.

What is the "thundering herd" problem?

A service has a two-second blip; **ten thousand clients** fail and all start a 4 s backoff at almost the same instant. Four seconds later **all retry simultaneously**, the just-recovered service is hit by a synchronised wall of traffic and falls over again; they back off 8 s and it repeats, harder. The clients have accidentally become a **denial-of-service attack**. **Jitter breaks the synchronisation** by spreading retries across a window. (The same effect on caches: § Cache stampedes.)

What does "idempotent" mean?

An operation is **idempotent** if performing it many times has the same effect as once: `set balance to 100` is; `add 100 to balance` is not (five times adds 500). It matters because a retry might be the *second* execution of something that already succeeded. (§ Delivery guarantees and idempotency.)

Part 1 — The problem

In a distributed microservices architecture **failures are inevitable**: unreliable networks, transient server **outages**, overloaded load balancers, bugs, operator mistakes. You can't build a system that never fails, so you design for **resilience** — a small hiccup mustn't snowball into an outage.

Transient failures specifically

Short-lived errors that resolve themselves given a little time — a network glitch, momentary loss of connectivity, temporary unavailability.

The classic example: a database connection is refused at the peak of a spike in **concurrent** users; **a few seconds later, after some connections close, a retry goes straight through.** Nothing was fixed; the condition passed. Other common cases: network **timeouts**, transient **HTTP 5xx** errors (500–599, "the server failed"; 4xx means "the request was wrong"), temporary resource exhaustion.

Why this needs solving

Without a strategy, services **propagate errors to end-users or upstream systems** (callers waiting on you) **unnecessarily** — a tiny blip triggers a user-facing error or manual intervention, though **the system might have recovered milliseconds later**.

Part 2 — The solution

If an operation fails for a temporary reason, **wait briefly and try again**, assuming the issue may have cleared. Distributed systems often fail **partially** (some requests fail, others succeed) or briefly, so the same request often simply succeeds second time — and behind a load balancer the retry **may hit a less-busy instance** entirely. Retries **mask sporadic failures**, raising the reliability users perceive.

Why it works

Retries assume the failure was transient or **non-deterministic** (not guaranteed to recur). Network blips clear, **threads** free up, services restart — so a later attempt has a good chance where the first failed. This sharply cuts the error rate seen higher up, and a service can complete the **downstream** call **without any manual fix**.

Part 3 — Transient vs. permanent: the critical distinction

Retry only errors **likely to be temporary**.

Worth retrying: network timeouts · HTTP 5xx · 502 Bad Gateway from a load balancer · database **deadlocks** (two transactions each waiting for the other's lock; the database kills one) · connection refused · **throttling / rate-limit** errors (the service deliberately refusing because you send too much)

Not worth retrying: HTTP 400 (validation failure) · HTTP 401 (**authentication** failure) · **business logic** errors (e.g. insufficient funds) · configuration errors · fatal **exceptions**

> **No amount of retrying fixes a bad request or invalid credentials** — those are permanent *for that request*, and retrying just repeats the failure and adds load.

So a good retry mechanism **checks the error or exception type**, retrying only transient conditions and letting permanent ones **fail fast**.

Retry vs. fail-fast

The alternative is **fail-fast**, typically via a Circuit Breaker:

	Retry	Fail-fast (Circuit Breaker)
Behaviour	Keeps trying, hoping for eventual success	Stops trying after detecting a pattern of failures
Favours	Eventual success	Conserving resources
Cost	Extra wait time and work	Gives up on things that might have worked
Right for	Transient faults	Long-lasting or permanent faults

For a service down for hours, blind retrying wastes time and resources; breakers or **graceful degradation** (defaults or **cached** data) fit better. **Retries for hiccups; fall back or fail fast for lasting errors** — complements, not competitors.

Part 4 — Architecture

The pattern is a **wrapper around an operation** that intercepts failures and repeats per a policy.

Retry policy and parameters

Defines **when and how**: **maximum attempts**, **which exceptions/error codes are retryable**, and the **delay strategy** — from *"3 times, 2 s apart"* to *"5 times, exponentially increasing delays with randomness, never on certain error types"*.

Retry loop (the invoker)

Calls the operation, catches failures, checks the policy, waits, calls again — until success or max attempts. If it never succeeds it **propagates the final failure** to its caller, often after logging it.

Backoff strategy

Avoid a **tight loop** (repeating as fast as the CPU allows):

Strategy	Behaviour	When
Constant delay	Always wait the same, e.g. 5s	Simple cases; predictable recovery time
Exponential backoff	Each wait longer than the last: 1s, 2s, 4s	The general default
Capped exponential backoff	As above with a maximum	Prevents absurdly long waits

Exponential backoff **eases load on struggling services** and raises the odds the issue has cleared by the next attempt. The cap matters: five doublings from 1 s is a 32 s wait; eight is over four minutes.

Jitter (randomization)

Clients that fail together **retry in unison** after the same interval — the thundering herd, **a second failure spike**. Jitter randomises each delay (e.g. somewhere between 2 and 4 s instead of exactly 4), **spreading bursts** so downstream load stays even. Variants — **full jitter**, **equal jitter**, **decorrelated jitter** — share one idea: retries **de-synchronized across clients**.

> Adding randomness on purpose seems odd, but it measurably reduces **contention** (many clients competing for one resource) and speeds recovery.

Circuit breaker integration

Retries handle transient failures; breakers handle persistent ones by cutting off calls after a threshold. A breaker **prevents a retry loop from endlessly hitting a downed service**: after, say, 50 straight failures it opens and calls fail immediately; after a **cooldown**, **half-open** lets a test call

through and success closes it. **The balance:** allow ~3 quick retries per request, but if many requests in a row fail — a bigger problem — open the circuit for a while. (Ordering the two in Resilience4j: pitfall 6.)

Idempotency and side effects

If an operation has **side effects** — **charging a card, sending an email, incrementing a counter** — a retry can perform it **multiple times** unless you design against it.

Retried operations should ideally be **idempotent**. Many **APIs** already are — deleting a resource, a `GET` (HTTP read). For non-idempotent actions (creating an order, taking a payment) use **idempotency keys**: a unique ID the client attaches to an operation and reuses on every retry, so the server can recognise a duplicate and not process it twice (implementation: Part 14D).

> The failure mode to picture: the payment succeeds on the server, but the response is lost. The client sees a timeout — indistinguishable from "it never happened" — and retries. **Without an idempotency key, the customer is charged twice.** The client cannot tell "lost request" from "lost response", which is why the **server** must **deduplicate**.

Placement in the architecture

- **Client side of a service-to-service call** — A's client library or **HTTP client** retries; B stays **stateless** and unaware, seeing only repeated requests.
- **API gateway or service mesh** — infrastructure retries automatically.

⚠️ **The key rule: never duplicate retries at multiple layers.**

> If every hop retries, **one failure multiplies into a storm**: three layers × three attempts = **27** requests from one original. **Centralize retries in one layer** — the edge service, or the immediate caller — disabling them in lower-level clients, or coordinate with a **token bucket** limiting overall retry rate (tokens refill at a fixed rate and each retry spends one, so retries can never exceed it).

Part 5 — Worked example (Java)

```java
public class RetryUtil {

    public static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
            long baseDelayMillis, double backoffFactor, boolean useJitter) throws Exception {

        int attempt = 1;
        Exception lastException = null;
        Random random = new Random();

        while (attempt <= maxAttempts) {
            try {
                return task.call();                 // success — return immediately
            } catch (Exception e) {
                lastException = e;                  // remember it for the final throw

                if (attempt == maxAttempts) {
                    break;                          // no retries left
                }

                // Work out how long to wait
                long delay = baseDelayMillis;
                if (backoffFactor > 1.0) {
                    // exponential: base * factor^(attempt-1)
                    delay = (long) (baseDelayMillis * Math.pow(backoffFactor, attempt - 1));
                }
                if (useJitter) {
                    // "full jitter": random value between 0 and the computed delay
                    delay = (long) (random.nextDouble() * delay);
                }
                // (a maximum cap could be applied here)

                try {
                    Thread.sleep(delay);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();   // restore the interrupt flag
                    break;                                // stop retrying
                }

                attempt++;
            }
        }
        throw lastException;    // all attempts exhausted
    }
}
```

The parameters

Parameter	Meaning
`maxAttempts`	Total tries **including the initial attempt**. `3` means 1 try + 2 retries
`baseDelayMillis`	Fixed delay if constant; the initial delay if exponential
`backoffFactor`	The multiplier. `2.0` with base 100ms gives 100ms, 200ms, 400ms. Set `1.0` for a constant interval
`useJitter`	Applies full jitter — a random value between 0 and the computed delay

Reading the logic

- `Callable<T>` (an operation that returns a value and may throw) makes it reusable for any network call or query.
- **Success returns immediately** — no further attempts or delay.
- **The exception is saved, not swallowed** — `lastException` is rethrown at the end so the caller learns *why*.

- **The attempt check precedes the delay** — no sleeping before giving up.
- **`Math.pow(backoffFactor, attempt - 1)`** — attempt 1 waits **`base × 1`**, attempt 2 **`base × factor`**, and so on.
- **The `InterruptedException` handling is correct and worth copying.** When a thread is **interrupted** (asked to stop), catching the exception clears the flag; **`Thread.currentThread().interrupt()`** restores it so code higher up still knows. Forgetting this is a classic Java bug that stops applications shutting down.

Using it

```
String result = RetryUtil.executeWithRetry(
    () -> fetchData(url),
    3,        // up to 3 attempts
    1000,     // 1 second base delay
    2.0,      // exponential backoff, doubling
    true      // use jitter
);
```

Up to 2 retries — about 1 s before the first, about 2 s before the second, randomised — throwing after roughly 3 s of total waiting. **Often one retry is all it takes** to ride out a glitch.

Design choices, and one to change

- **Generic `Callable<T>`** — reusable across operation types.
- **Configurable backoff and jitter** — constant (`1.0`, `false`), exponential (`2.0`, `false`), or exponential + jitter (`2.0`, `true`).
- **`maxAttempts` prevents infinite loops** that would endlessly hammer a failing service.
- **Change before production:** it **catches broad `Exception`**, so *any* failure is retried — including validation errors. Catch **only specific transient exceptions** (`IOException`, timeouts) and let others bubble up; **your error filtering belongs here**.

Jitter refinement: full jitter (`random × delay`) can yield a near-zero wait, partly defeating backoff. Many implementations randomise between **50% and 100%** of the delay ("equal jitter") to keep a minimum gap.

Worth adding: a log line and **metric** per retry, and a check that aborts retrying if a higher-level circuit is already open.

Part 6 — Performance implications and risks

Increased load on dependencies

Every retry is **another request** — negligible when failures are rare. **But** if a service is failing *because it's overloaded*, retries worsen it; many clients retrying a struggling service create a **retry storm that amplifies load exponentially**. Hence **backoff**, **limited attempts**, and breakers or throttling layered on top.

Increased latency for end users

Retries lengthen response time. Sometimes acceptable — **a slightly slower success beats an immediate error** — but overdone, and chained across services, they degrade the experience. Countermeasure: **an overall timeout for the operation** — e.g. succeed **within 5 s including all retries** or not at all — to keep a responsive **SLA** (service-level agreement: promised speed or uptime). Otherwise retries silently blow your **latency budget** (the maximum time a request may take).

Wasted work and resource consumption

A retry that ultimately fails wasted CPU, **bandwidth** and memory; if the issue isn't transient, *all* retries are waste. Detect non-transient failures quickly — "permission denied" or "invalid request" won't improve — with a **blacklist of non-retryable errors**. Also: **if the first attempt already took**

nearly the full client timeout, another attempt will likely time out too; propagate the error instead of tying up threads.

Cascading effects in complex systems

A → B → C, each with its own policy: C fails, B retries, *and* A retries its call to B — multiplying load on B and C. Decide **which layer retries** and let others fail fast; commonly the edge/client-facing service retries and intermediate services propagate errors. Monitor for excessive retries — they often signal a downstream **incident**.

Retry storms and thundering herd

When a popular service glitches, many services retry at once. Backoff and jitter avoid synchronisation — **jitter specifically tackles the thundering herd**. Also: **retry budgets** (a cap on retries per second) and **adaptive algorithms** that lengthen backoff while errors continue.

Idempotency revisited

Duplicate side effects cost performance too — the same email sent twice uses twice the resource. **Never retry non-idempotent operations blindly** without deduplication keys or **check-pointing** (recording progress so a repeat skips completed steps).

The mitigation checklist

- **Exponential backoff** — don't hammer
- **Jitter** — avoid synchronised retries
- **Limit attempts**, and total time spent retrying
- **Circuit breakers** / fail-fast where failure is likely persistent
- **Distinguish error types** — retry only transient ones
- **Monitor and tune** — frequent retries signal a struggling dependency or a bad policy
- **Ensure idempotency**, or server-side deduplication for critical actions
- **Test under failure** — simulate a dependency down or slow and verify retries improve resilience rather than overwhelm the system

---

Part 7 — Use cases

- **Web services** — the internet is unreliable and the **cost asymmetry** favours retrying: a failed request is costly (a user-facing error or expensive fallback), a retry is cheap (one more request).
- **Database operations** — network issues, concurrency conflicts, temporary unavailability. Clearest case: you save a record while **another user is updating it**; retry after a short delay lets the conflicting **transaction** finish and yours succeed.
- **Microservice architectures** — retrying gives a failing dependency **a chance to recover**, so a hiccup doesn't become a full outage.

Part 8 — Real-world examples

Cloud SDKs (AWS, Google Cloud). (An **SDK** is a platform's official client library.) AWS SDKs automatically retry transient 5xx and throttling errors with **exponential backoff and jitter**; Google Cloud libraries retry idempotent requests (a timed-out storage upload retried behind the scenes). **Developers benefit without even realising it.**

Netflix. Internal libraries retry calls to services or **data stores** when a request fails or a **host** is unresponsive — **retrying on another instance**, perhaps after a brief backoff, rather than erroring. With circuit breakers, sporadic failures don't disrupt viewing.

Payment processing (Stripe, PayPal). Financial APIs hit transient failures talking to banking networks, so they retry behind the scenes — and **encourage idempotency keys** on requests, so a client can safely

retry with the same key and the server **won't duplicate the charge**. The API **explicitly assumes clients will retry**: a glitch between **merchant** and **gateway** (the payment provider's service) doesn't lose the order or double-charge.

Message queues and event processing. In **RabbitMQ** or **Kafka** (message brokers), retry means **requeueing** or **dead-letter queues** (parking messages that keep failing). A **consumer** that hits a temporary error doesn't **acknowledge** the message, so it returns to the queue for a later attempt — retry in an **asynchronous** context. E.g. reserving inventory fails on a **lock timeout** (waited too long for another transaction's row lock); retried a minute later, the lock may be free. Many e-commerce platforms rely on this in **event-driven architectures** for **eventually consistent** outcomes (Event-Driven).

Part 9 — System design examples

E-commerce checkout. The user clicks Buy, but the **payment gateway** (the third-party service charging cards) **is temporarily unavailable**. Rather than reporting failure, retry and handle the glitch gracefully — and this is exactly where idempotency keys are non-negotiable (the double-charge scenario).

IoT data collection. IoT (Internet of Things) devices send readings periodically, but the **database is temporarily unavailable** (network glitch, scheduled maintenance). Rather than discard data, retry to **ensure nothing is lost**.

Part 10 — When NOT to use retry

- The error is clearly **non-transient** (configuration error, fatal exception) — retrying only delays the inevitable
- The service is **known to be down for an extended period** (planned outage) — use a breaker or fallback
- **Strict latency requirements** — **real-time** systems may prefer failing fast to violating their SLA
- The operation is **extremely expensive**, or has **side effects that can't safely repeat**

Alternatives: **manual compensation** (undoing effects), **eventual reconciliation** (comparing records later and fixing mismatches) for async processes, or **alerting a human**.

Part 11 — Common beginner misconceptions

- **"More retries means more reliability."** Only up to a point; after a few attempts you mostly add latency and load. If three with backoff failed, the fourth probably will too.
- **"Retry and circuit breaker are alternatives."** Complements doing opposite things: retry handles the blip, the breaker the outage.
- **"Jitter is a micro-optimisation."** At ten thousand clients it's the difference between graceful recovery and a self-inflicted outage.
- **"My operation is a GET, so idempotency doesn't matter."** Check the whole path: a GET that triggers a counter increment or **audit write** isn't as idempotent as it looks.
- **"Retries are free if they usually succeed."** They're free when failures are rare — and most expensive exactly when the system is struggling, which is when they fire most.
- **"I'll retry at every layer to be safe."** The most damaging mistake here: layered retries multiply into an exponential storm.

Part 12 — Check your understanding

1. Why is retrying an HTTP 401 pointless?
2. Explain the thundering herd, and how jitter fixes it.

3. Why can't a client distinguish "lost request" from "lost response," and what follows from that?
4. Why must you set an overall operation timeout alongside a retry policy?
5. If A retries B and B retries C, how many requests can one failure become at 3 attempts each?
6. What's the one line in the example code you should change before production use?

(Answers: 1 — authentication failure is permanent for that request; no number of attempts fixes wrong credentials; 2 — many clients fail simultaneously, back off by the same amount, and retry in unison, re-overloading the service; jitter randomises delays so retries spread out; 3 — both look like a timeout, so the server must deduplicate using an idempotency key rather than the client guessing; 4 — otherwise cumulative retry delays silently exceed your latency budget; 5 — nine, since each of A's 3 attempts triggers 3 of B's; 6 — catching broad `Exception`, which retries permanent errors too.)

Part 13 — Interview soundbite (30 seconds)

> "Retry handles transient failures — network blips, timeouts, throttling, connection refused during a spike — things that fix themselves in seconds. You wrap the call in a policy: max attempts, which errors are retryable, and a delay strategy. Two details matter most: exponential backoff, so you're not hammering a service that may be failing precisely because it's overloaded; and jitter, because otherwise every client that failed together retries together and the thundering herd causes a second outage. You need idempotency: the client can't distinguish a lost request from a lost response, so retrying a payment without an idempotency key double-charges — which is why Stripe's API is built around them and assumes clients retry. Distinguish transient from permanent: retrying a 400 or 401 is waste. The biggest architectural mistake is retrying at every layer — if A retries B and B retries C, one failure becomes nine requests — so pick one layer and let the rest fail fast. Pair it with circuit breakers — retry for hiccups, the breaker for outages — and set an overall operation timeout so retries can't blow the latency budget."

Part 14 — Implementation

A. Runnable demo — backoff, jitter, and the thundering herd

Full source: `demos/chapters/08-retry/RetryDemo.java`

```
cd demos && ./gradlew runCh08
```

The demo's `executeWithRetry` is Part 5's utility with the three production fixes applied — a separate catch for permanent errors, a **cap**, and **equal jitter** (50–100% of the delay):

```
} catch (PermanentException e) {
    System.out.println("      permanent error, NOT retrying: " + e.getMessage());
    throw e;                                  // fail fast
} catch (Exception e) {
    last = e;
    if (attempt == maxAttempts) break;

    long delay = (long) (baseDelayMs * Math.pow(factor, attempt - 1));
    delay = Math.min(delay, 5000);            // cap
    if (useJitter) {
        // "equal jitter": 50-100% of the computed delay
        delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
    }
    ...
```

`main` runs: a `FlakyService` that throws `TransientException("timeout")` twice then succeeds (5 attempts, base 100 ms, factor 2); a `PermanentException("HTTP 401")`; the same backoff schedule against a permanent

`503` with and without jitter; and a herd simulation that computes 1,000 clients' delay (base 1,000 ms, optionally jittered) and reports the busiest 100 ms bucket.

B. What you should see

```
=== 1. Transient failure, recovers on 3rd attempt ===
      attempt 1 failed (timeout), waiting 100ms
      attempt 2 failed (timeout), waiting 200ms
      succeeded on attempt 3
      result: OK after 3 calls

=== 2. Permanent failure — should NOT retry ===
      permanent error, NOT retrying: HTTP 401
      gave up immediately (correct)

=== 3. Backoff schedule, no jitter ===
      attempt 1 failed (503), waiting 100ms
      attempt 2 failed (503), waiting 200ms
      attempt 3 failed (503), waiting 400ms
      attempt 4 failed (503), waiting 800ms      <- doubling

=== 4. Same schedule WITH jitter ===
      attempt 1 failed (503), waiting 74ms
      attempt 2 failed (503), waiting 133ms
      attempt 3 failed (503), waiting 291ms      <- randomised
      attempt 4 failed (503), waiting 512ms

=== 5. Thundering herd: 1000 clients retry at once ===
      jitter=false -> busiest 100ms bucket holds 1000 of 1000 clients
      jitter=true  -> busiest 100ms bucket holds ~205 of 1000 clients
```

Section 5 is the one to internalise. Without jitter **all 1000 clients retry in the same 100ms window** — a synchronised wall hitting a service that just recovered. With jitter the worst bucket holds about a fifth. Same number of retries, spread over time instead of stacked.

C. Production — Resilience4j Retry

(Library setup: Circuit Breaker 16C.)

```yaml
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3                  # 1 initial + 2 retries
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true      # <- jitter
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - org.springframework.web.client.HttpServerErrorException   # 5xx
        ignoreExceptions:
          - org.springframework.web.client.HttpClientErrorException   # 4xx — never retry
```

```java
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
@CircuitBreaker(name = "inventoryService")     // order matters — see pitfall 6
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}
```

Spring's own `@Retryable` is an alternative without Resilience4j:

```java
@Retryable(
    retryFor = { IOException.class },
    noRetryFor = { ValidationException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 200, multiplier = 2, random = true))
public Inventory check(String sku) { ... }

@Recover
public Inventory recover(IOException e, String sku) { return Inventory.unknown(sku); }
```

D. Implementing idempotency keys

The most important production concern, usually missing from tutorials — and the book's reference implementation (Saga steps and event consumers apply the same idea with a saga ID or event ID). The client sends a stable key — usually a **UUID** (random, practically unique 128-bit ID) in an HTTP **header**; the server guarantees one execution per key.

Client:

```java
String idempotencyKey = UUID.randomUUID().toString();   // generated ONCE, before the first attempt

executeWithRetry(() ->
    http.post("/payments")
        .header("Idempotency-Key", idempotencyKey)      // same key on EVERY retry
        .body(payment)
        .execute(),
    3, 200, 2.0, true);
```

> **Critical detail:** generate the key **outside** the retry loop. Generated inside, each attempt looks brand-new and the customer is charged three times — the very bug the mechanism exists to prevent.

Server:

```java
@PostMapping("/payments")
public ResponseEntity<PaymentResult> pay(
        @RequestHeader("Idempotency-Key") String key,
        @RequestBody Payment payment) {

    // Atomic "insert if absent" — the DB unique constraint does the work
    Optional<PaymentResult> existing = idempotencyStore.find(key);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get());     // replay the original response
    }

    PaymentResult result = paymentService.charge(payment);
    idempotencyStore.save(key, result, Duration.ofHours(24));
    return ResponseEntity.ok(result);
}
```

Store keys in **Redis** with a **TTL** (expiry), or a table with a **unique index** on the key (the database rejects a second row with the same value). The **unique constraint** is what makes it safe under concurrent duplicate requests.

E. Implementation pitfalls

1. Retrying non-idempotent operations without a key. The headline risk: double charges, duplicate orders, duplicate emails.

2. Generating the idempotency key inside the retry loop. Silently defeats the mechanism (above).

3. Retrying 4xx errors. A 400 or 401 fails identically every time. Always configure `ignoreExceptions`.

4. No cap on exponential backoff. Base 1s × 2 over 10 attempts ends at a **17-minute** wait. Always set a maximum.

5. No overall deadline (one limit for the whole operation, all attempts included). A 5s timeout × 3 attempts plus backoff is potentially 20 seconds. Budget the whole operation:

```
resilience4j.timelimiter.instances.inventoryService.timeoutDuration: 3s
```

6. Wrong ordering of retry and circuit breaker. Resilience4j's default is **Retry(CircuitBreaker(call))** — the breaker is *inside*, so it sees each attempt. Usually what you want. Inverted, the breaker counts a whole exhausted retry sequence as one failure and opens far too slowly.

7. Retrying at every layer. HTTP client 3× × service 3× × gateway 3× = **27** requests to an already-struggling service. Pick one layer and explicitly disable the rest — most HTTP clients retry by default.

8. Retrying immediately with no delay. A tight retry loop against a struggling service is indistinguishable from an attack.

9. Not logging retries. Invisible retries make a dependency degrading from 0.1% to 15% failure look healthy until it tips over. Emit a metric per retry.

Related patterns

- **Circuit Breaker** — the essential companion; stops retries becoming an attack on a downed service
- **Bulkhead** — bounds the resources retries can consume
- **Service Discovery** — retry against a *different* discovered instance
- **Saga** — compensation for when retrying isn't safe or sufficient
- **Event-Driven Architecture** — requeueing and dead-letter queues are retry in an async context